# File permissions in Linux

## Project description

In this activity, I examined file and directory permissions in Linux for the research team's projects directory. I used Linux commands to check current permissions, identify unauthorized access, and update permissions using chmod. This helps ensure that only authorized users have the correct access to files and directories.

## Check file and directory details

To review the existing permissions in the projects directory, I used the following command:

ls -la

The `ls` command lists directory contents. The `-l` option displays detailed information about files and directories, while the `-a` option displays hidden files. This command allowed me to view all files, directories, and their associated permissions.

The output showed the following permissions:

-rw-rw-rw-  project_k.txt

-rw-r-----  project_m.txt

-rw-rw-r--  project_r.txt

-rw-rw-r--  project_t.txt

-rw--w----  .project_x.txt

drwx--x---  drafts

The first column of the output contains the 10-character permission string assigned to each file or directory.

## Describe the permissions string

The 10-character permissions string identifies both the file type and the permissions granted to users.

For example:

-rw-rw-r--

The characters represent the following:

- 1st character: Indicates the file type. A hyphen (-) means the item is a regular file, while d indicates a directory.
- 2nd–4th characters: User permissions (read, write, execute).
- 5th–7th characters: Group permissions (read, write, execute).
- 8th–10th characters: Other permissions (read, write, execute).

In this example, the user has read and write permissions, the group has read and write permissions, and others have read-only permission.

## Change file permissions

The organization does not allow others to have write access to any files. After reviewing the permissions, I identified that `project_k.txt` granted write access to others.

To remove this permission, I used the following command:

chmod o-w project_k.txt

The `chmod` command changes file permissions. The `o-w` option removes write permissions from others.

After running the command, the permissions became:

-rw-rw-r-- project_k.txt

I verified the change by running:

ls -la

## Change file permissions on a hidden file

The file `.project_x.txt` is a hidden file because its name begins with a period (`.`). The research team archived this file and required that neither the user nor group have write access. However, both should retain read access.

To update the permissions, I used:

chmod u-w,g-w,g+r .project_x.txt

This command:

- Removes write permission from the user
- Removes write permission from the group
- Adds read permission to the group

After the update, the file permissions became:

-r--r----- .project_x.txt

I verified the change using:

ls -la

## Change directory permissions

The organization requires that only researcher2 have access to the `drafts` directory and its contents. During my review, I found that the group had execute permissions on the directory.

To remove this access, I used:

chmod g-x drafts

This command removes execute permissions from the group while preserving access for the owner.

After the update, the directory permissions became:

drwx------ drafts

I verified the modification using:

ls -la

## Summary

In this activity, I reviewed file and directory permissions within the projects directory using the `ls -la` command. After identifying permissions that did not meet organizational security requirements, I used the `chmod` command to update permissions on files, hidden files, and directories. These changes ensured that access was restricted to authorized users and that the research team's data remained secure.